Programmeringsspråket Java

Syntax i Java

Java

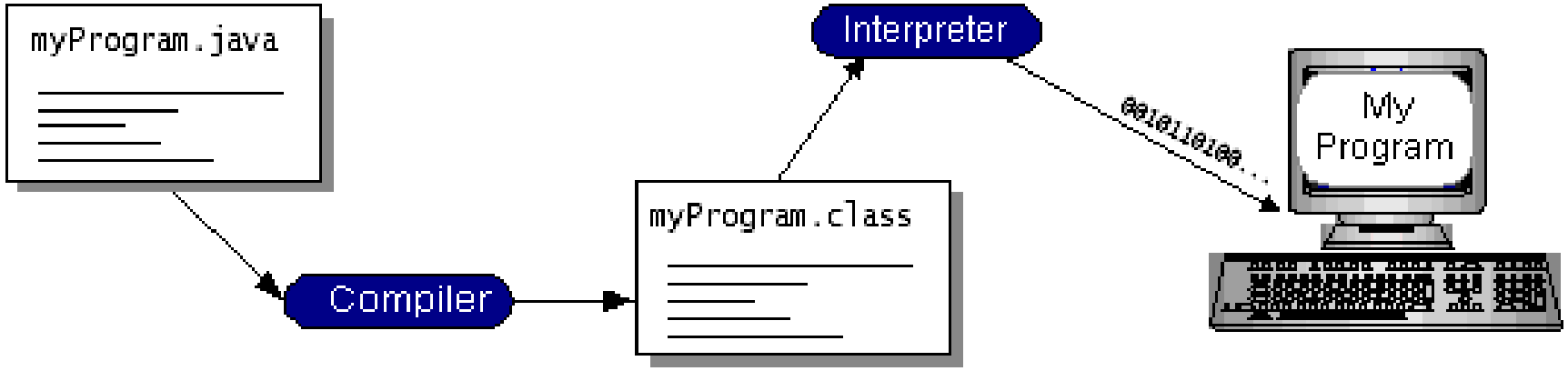
- Objektorienterat
- Plattformsoberoende
  - ”write once – run anywhere”
- Språkets kärna (keywords) är liten
  - C-liknande syntax
- Javas API, Application Programming Interface
  - Stort antal färdiga klasser tillgängliga för t.ex. grafiska användargränssnitt, nätverksprogrammering, …

Java

- PC, mobil, smartphones, servrar, routrar, inbyggda system som t.ex. digitalboxar, …
  - ca 3 miljarder enheter kan exekvera Java
- Flera olika varianter av Java
  - *Java SE*, standard edition – terminal och desktop
  - *Java EE*, distribuerade system, ofta med databas och webb-ui
  - Android, applikationsprogrammering i för Android OS
  - *Java ME*, mobiltelefoner och andra ”embedded devices”
  - *Java Card*, appliktioner för smarta kort, ex. betalkort
- Skapades av SUN Microsystems, ägs numera av Oracle (gäller ej Android)
  - Öppen specifikation och (i många fall) källkod
  - Flera parallella implementationer: Oracle, Open JDK, …

Plattformsoberoende

- Källkod skriven i Java kompileras till s.k. java-bytekod
  - *ej* maskinkod
- Java-bytekod kan exekveras av en applikation kallad ”java-tolk”, ”virtuell maskin”, JVM eller JRE
- Tolken/JVM är specifik för olika plattformar, men kan exekvera bytekod oavsett var denna kompilerats



Plattformsoberoende

- JVM en del av datorvärldens ”ekosystem”
  - Installerad på i princip alla datorer
- Interpretering ger långsammare exekvering än ren maskinkod, men
  - dagens virtuella maskiner är mycket effektiva
  - JIT, Just In Time-compilation – JVM genererar maskinkod under exekveringen i vissa fall
- Det finns också kompilatorer som direkt genererar maskinkod
  - Denna kod är då plattformsberoende

Vad behövs för att skriva Javaprogram?

- Java SE Software Development Kit, JDK
  - Hjälpmedel för att utveckla, felsöka och kompilera javakod
- Innehåller bl.a.
  - Kompilator, javac.exe
  - Virtuell maskin, java.exe, med tillbehör (JRE)
  - Javas API, standardklasser som kan användas i alla Javaprogram
- Utvecklingsmiljö, IDE
  - IntelliJ, VS Code, Netbeans ,Eclipse, …

Ett enkelt Javaprogram

```
import java.util.Scanner;

public class Welcome {

	public static void main(String[] args) {

		Scanner scan = new Scanner(System.in);

		System.out.print("Hi, what's your name? ");
		String name;
		name = scan.next();

		System.out.println("Welcome to Java, " + name);

		scan.close();
	}
}
```

Kompilering och exekvering i terminal

- Spara filen som ”ClassName.java”, här ”Welcome.java”

- Kompilera:
  ```
  > javac Welcome.java
  ```

  Bytekod-filen ”Welcome.class” genereras

- Exekvera:
  ```
  > java Welcome
  ```

  Bytekoden tolkas av virtuella maskinen, JVM

- ```
  > Hi, what's your name? Anders
  > Welcome to Java, Anders
  >
  ```
  ```

Javasyntax

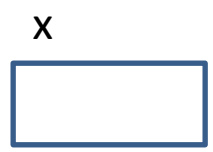
- Stora likheter mellan syntaxen i Java och C
- Dessutom klasser
  - All(!) kod skrivs i klasser
- Klasser = sammansatta datatyper
  - mallar för objekt
  - jämför med *struct* i C
- Objekt = variabler av klasstyp med
  - datamedlemmar som representerar tillståndet
  - medlemsfunktioner, metoder, som utför operationer på data

Datatyper: primitiva, klass, enum

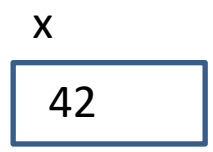
- Primitiva datatyper, alla ”signed”
  - `byte, short, int (4 byte), long (8 byte),`
  - `float (4 byte), double (8byte)`
  - `boolean: true eller false`
  - `char: 2 byte - unicode`
- Klasser
  - Sammansatta datatyper
- Enum
  - Definierar en begränsad uppsättning värden

Primitiva datatyper

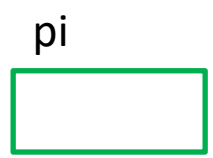
- Deklarera en variabel som kan lagra data av typen heltal:
  ```
  int x;
  ```



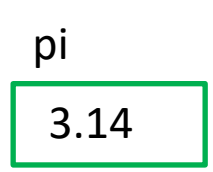
- Tilldela variabeln ett värde:
  ```
  x = 42;
  ```



- Deklarera en variabel för som kan lagra data av typen flyttal:
  ```
  double pi;
  ```



- Tilldela variabeln ett värde:
  ```
  pi = 3.14;
  ```



Primitiva datatyper

- ”Scope” – en variabel existerar endast inom det kodblock där den deklarerats
- {
  int x; …;
  x = 42; …;
  }
  // Variabeln x existerar inte längre, ”out of scope”

- ”=”, tilldelning, kopierar alltid variabelns värde i Java
  int a = 42;
  int b = a;

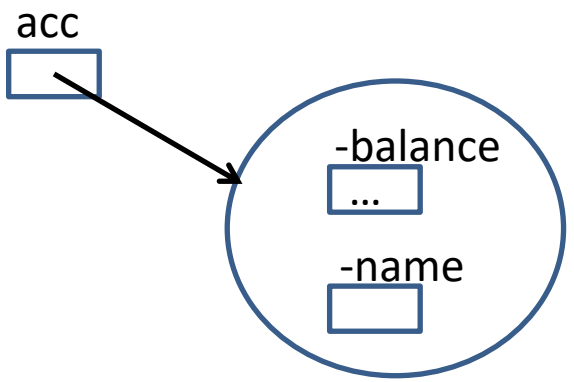


Klasstyper, referenser vs objekt

- Instanser av klasstyp, dvs. objekt, hanteras via referensvariabler
  - Referenser är som pekare, men säkrare – kan ej adressera direkt
  - En referensvariabel kan referera ett objekt eller ha värdet null:
    Account acc = null; // en referens som (ännu) inte refererar något objekt.

Klasstyper, referenser vs objekt

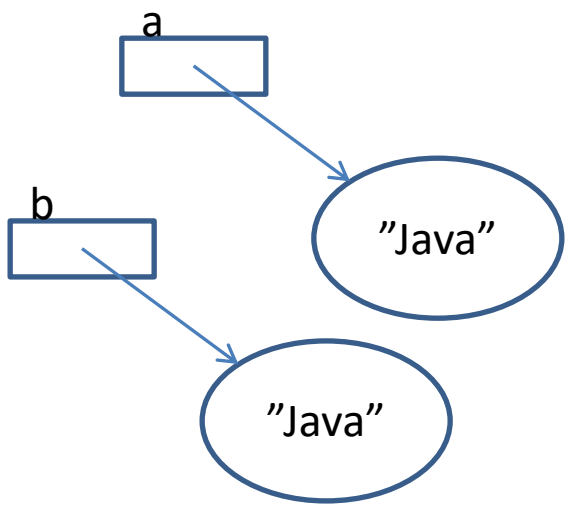
- Account acc;
- acc = new Account(”Anders”);



- ”acc” är en referensvariabel
  - Kan referera (”adressera”) ett objekt, eller…
  - …ha värdet ”null”
- ”new” tillsammans med konstruktor skapar ett nytt objekt och initierar detta
- ”=” tilldelar acc adressen till det nya objektet

Klasstyper, referenser vs objekt

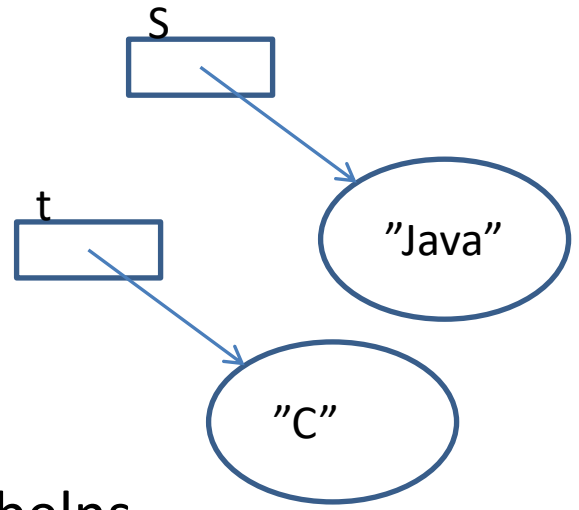
- Skilj på referensvariabeln och det objekt som refereras
- String a = new String(”Java”);
  String b = new String(”Java”);



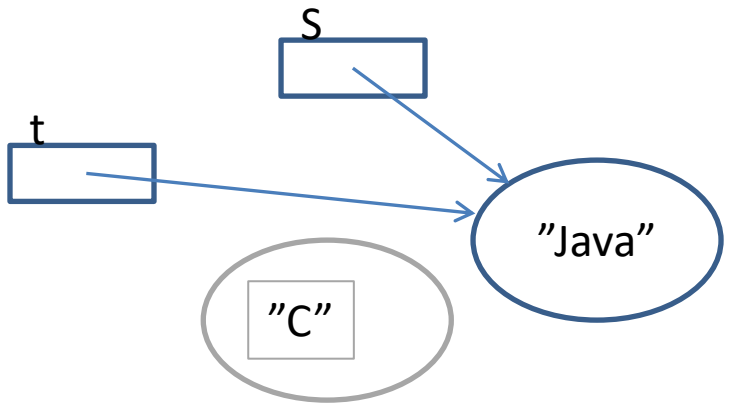
- Jämförelse med ”==” innebär jämförelse av referensernas värden, ”adresser”
- Jämförelsen ”a == b” ger i detta fall värdet false(!)
  - ”==” jämför om samma objekt refereras, men referenserna har i detta fall olika värden
  - Använd a.equals(b), eller liknande metod, för att jämföra data i objekten

Klasstyper, referenser vs objekt

- String s = new String(”Java”);
  String t = new String(”C”);



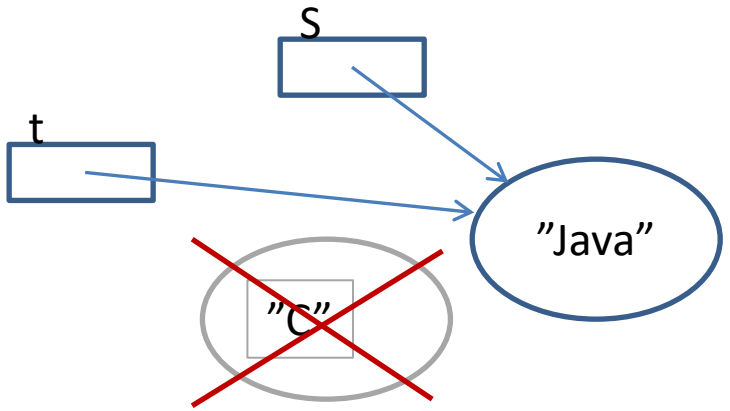
- ”=”, tilldelning, kopierar alltid variabelns värde i Java, men i detta fall är värdet en referens, ”adress”
- t = s;
  // t and s are aliases
- Ingen referens till objektet ”C”



Garbage Collection, GC

- En referensvariabels livslängd, ”scope”, är till slutet av det kodblock där den deklarerats, men …
- Ett objekts livslängd är (i Java) från det objektet skapas med ”new” tills det inte längre refereras av någon referensvariabel

- Objektet deallokeras från minnet av en en s.k. ”Garbage collector”
  - Sker inte omedelbart, GC söker med jämna(?) mellanrum igenom minnet efter objekt som kan deallokeras
  - Korta ”hick ups” under exekveringen …



- I språk som C och C++ måste programmeraren hantera denna de-allokering av dynamiskt allokerat minne (anropa malloc eller destruktor)
  - Vanlig källa till minnesläckor

”var”

- Account acc = new Account(”Anders”);
  kan ersättas av
  var acc = new Account(”Anders”);
  - Objektets typ bestämmer referensvariabelns typ
- Endast för lokala variabler
- Fr.o.m. version 10 av java SE

Javasyntax

- Styrande satser och villkor:
  `if`, `while`, `do-while`, `for` och `switch`
  som i C
- Logiska operatorer:
  `&&`, `||`, `!`
- Alla villkorsuttryck är av typen `boolean`, beräknas till true eller false

Logiska uttryck i Java

- ```
  int temperature = …;
  boolean cold = temperature < 18;

  if(cold) {
     System.out.print(”Bless you”);
  }
  ```

- ```
  boolean rich, sick;
  …;

  if(rich && !sick) {
     System.out.print(”Lucky you”);
  }
  ```

Javasyntax, villkor och iteration

```
int testscore = 76;
char grade;

if (testscore >= 90) {
        grade = 'A';
}
else if (testscore >= 80) {
        grade = 'B';
}
else if (testscore >= 70) {
        grade = 'C';
}
else if (testscore >= 60) {
        grade = 'D';
}
else {
        grade = 'F';
}

System.out.println("Grade = "
+ grade);
```

Alternativ form för switch i Java

En switch-sats kan i Java skrivas på samma sätt som i C:

```
int number = 42;
String numberStr;
switch(number) {
    case 0: numberStr = "zero"; break;
    case 1: numberStr = "single"; break;
    case 2:
    case 3: numberStr = "few"; break;
    default: numberStr = "many";
}

System.out.println(numberStr);
```

I Java kan ett switch-block returnera ett värde. Med denna syntax är inte break nödvändigt.

```
String numberStr = switch(number) {
    case 0 -> "zero";
    case 1, 2 -> "single";
    default -> numberStr = "many";
};

System.out.println(numberStr);
```

Javasyntax, villkor och iteration

```
double amount = 1000.0;
int years = 8;
int counter = 0;

while( counter < years) {
        amount = amount*1.02;
        counter = counter + 1;
}

System.out.println(
        ”amount= " + amount);
```

```
double amount = 1000.0;
int years = 8;
int counter = 0;

while( counter < years) {
        amount *= 1.02;
        counter++;
}

System.out.println(
        ”amount= " + amount);
```

Javasyntax, villkor och iteration

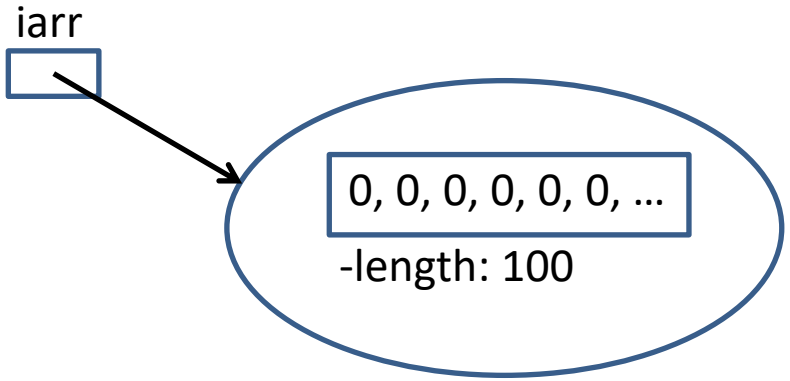
```
double amount = 1000.0;
int years = 8;

for( int counter = 0; counter < 8; counter ++) {
        amount *= 1.02;
}

System.out.println(
        ”amount= " + amount);
```

Javasyntax, arrayer

- Arrayer är objekt som känner till sin egen längd – datamedlemmen ”length”
- `int[] iarr = new int[100]; // zeros`

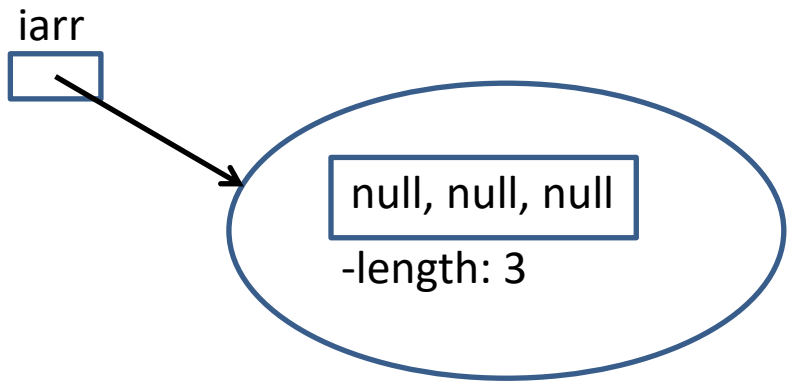



- ```
  for(int i=0; i < iarr.length; i++) {
      iarr[i] = 10*i;
  }
  ```

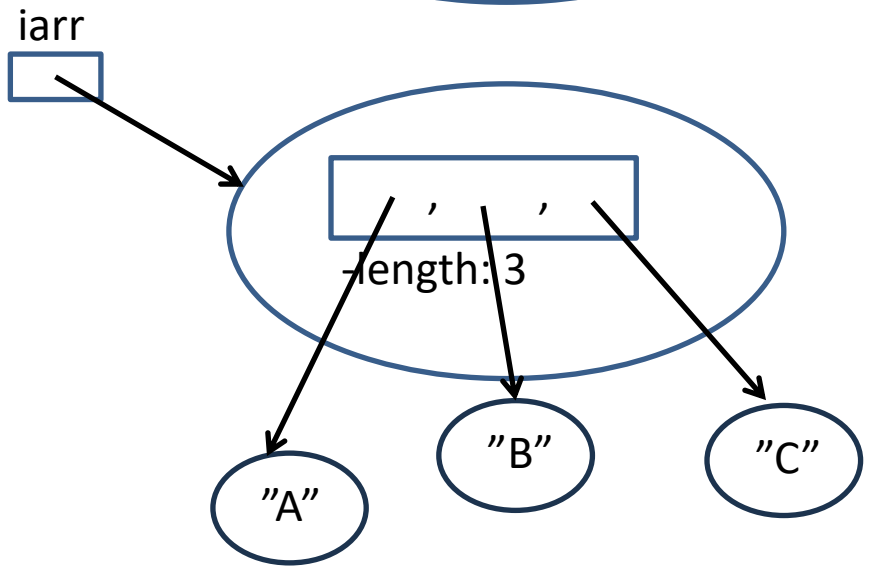
- Kontroll av felaktig av indexering under exekvering
  - ArrayIndexOutOfBoundsException
- Automatisk initiering av fälten: 0, false eller null
  ```

Javasyntax, arrayer med objektreferenser

- Arrayer för objekt innehåller endast referenser till objekten
- String[] sarr = new String[3]; // null values



- sarr[0] = new String(«A»);
  sarr[1] = new String(«B»);
  sarr[2] = new String(«C»);



- Nu är elementen i arrayen satta att referera objekt
  - Dessa objekt ligger på andra ställen i minnet

”for-each”-loop

```
int[] iarr = new int[100];
...
int sum = 0;
for(int value : iarr) {
        sum += value;
}
```

- Ger, element för element, kopior av värdena eller referenserna i arrayen
  - Kan inte ändra på ursprungliga värdet i arrayen
- ”for-each” fungerar med arrayer och med listor

enum

- En egen typ där ett begränsat antal värden definieras
- public enum Suit {
  HEARTS, SPADES, DIAMONDS, CLUBS
  }
- Suit s = Suit.CLUBS;
- if(s == Suit.CLUBS) { …
- Kompilatorn kontrollerar att endast definierade värden används - mindre risk att använda felaktiga värden
  - public void setTrump(int suit) { … // all int values valid
  - public void setTrump(Suit suit) { … // only HEARTS, …

Paket

- Ett paket är en samling klasser som hör ihop
- Katalog i filsystemet
- - Gruppera relaterade klasser
  - Mindre risk för namnkollisioner
  - Styra åtkomsten av data mellan klasser
- `import java.util.Random; // En klass`
- `import java.util.*;`
  `// Alla klasser i paketet`
- Dokumentation, Javas API:
  https://docs.oracle.com/en/java/javase/17/docs/api/index.html

Kommentarer – skilj på ”hur” och ”vad”

- ”Hur-kommentarer”, beskriver hur något utförs, ex. en knepig algoritm

```
//  to end of line
/*
   multiple lines
   …
*/
```

- Var sparsam med ”hur-kommentarer”
  - Källkoden ska vara självförklarande, med beskrivande namn på variabler, metoder och klasser
- ”Vad-kommentarer”, javadoc kommentarer, ska förklara *vad* en klass representerar eller vad en publik metod gör

```
/**
 * Documentation of classes
 * and (public) methods
*/
```

- Det ska vara möjligt att förstå hur en klass eller metod ska användas utifrån javadoc-kommentarerna, utan att läsa källkoden